Git & GitHub - Quick Revision Notes

Learn the idea → try the command → check with `git status`.

Examples use `main`. Replace names and URLs with your own values.

1. The basics

- **Git:** A distributed version control system that records project history.
- **GitHub:** A platform for hosting Git repositories and collaborating through pull requests, reviews, and issues.
- **Repository (repo):** Project files plus their Git history and metadata.
- **Commit:** A recorded snapshot of staged content, identified by a commit hash.
- **Branch:** A movable pointer to a commit; it is not a separate directory.
- **HEAD:** Usually points to your current branch. Checking out a specific commit can produce a detached HEAD.

Version control	Where history lives	Offline work
Centralized, e.g. SVN	Central server holds authoritative history	Can edit local files; commits normally require the server
Distributed, e.g. Git	A normal full clone has project history locally	Can edit, commit, branch, and inspect history offline

Remember: Reconnecting does not automatically sync a Git repo. Use fetch/pull and push.

The four places your work lives

Place	Purpose	Next action
Working directory	Files you edit	`git add` selects content
Staging area (index)	Snapshot prepared for the next commit	`git commit` records it
Local repository	Committed history on your computer	`git push` sends commits
Remote repository	Repository hosted elsewhere, e.g. GitHub	`git fetch` downloads history

Files: Untracked = not yet tracked; modified = changed; staged = selected for the next commit.

Set up once

```
git --version
git config --global user.name "Shreyash Agrawal"
git config --global user.email "YOUR_EMAIL"
git config --global init.defaultBranch main
```

The name and email identify your commits; they do not log you into GitHub. Use a GitHub-provided no-reply email if you prefer.

2. Everyday commands & ignored files

Command	Meaning
`git init`	Initialize a repo; files are not automatically tracked
`git clone URL`	Create a local copy of an existing repo
`git status`	Check the branch, staging area, and changed files
`git add file.txt`	Stage the current content of one file
`git add .`	Stage additions, modifications, and deletions under the current directory
`git add -A`	Stage changes across the whole repo
`git commit -m "message"`	Record the staged snapshot locally
`git diff`	Compare tracked working files with the staging area
`git diff --staged`	Compare staged content with the last commit
`git log --oneline -5`	Show the latest five commits
`git log --oneline --graph --all`	Show branch history as a graph
`git show HASH`	Inspect a commit and its changes

Important: If you edit a file after staging it, run `git add` again to include the new edits. A commit does not automatically include every working change.

A small workflow

```
git status
git diff
git add README.md
git diff --staged
git commit -m "Add Git revision notes"
```

.gitignore

Create a file named `.gitignore` and list patterns for local files that should remain untracked:

```
.env
node_modules/
*.log
```

Commit `.gitignore` so others use the same rules. It does **not** stop tracking files already committed.

```
git rm --cached .env
git add .gitignore
git commit -m "Stop tracking local environment file"
```

This keeps `.env` locally, but does not remove it from earlier commits. If secrets were published, revoke or rotate them; ignoring the file is not enough.

Empty directories: Git tracks files, not empty directories. Add a placeholder such as `.gitkeep` if needed; that name is a convention, not a Git feature.

3. Branching, merging & rebasing

Command	Meaning
`git branch`	List local branches; * marks the current one
`git branch feature-x`	Create a branch without switching
`git switch feature-x`	Switch to an existing branch
`git switch -c feature-x`	Create and switch in one step
`git branch -d feature-x`	Delete a local branch after Git's merge safety check

Uncommitted changes live in the working directory/index. Switching may carry them along, or Git may refuse if the switch would overwrite them. Commit or stash first when you want clean separation.

Merge = combine histories

To integrate `feature-x` into `main`:

```
git switch main
git merge feature-x
```

The **current branch** receives the merge; you can merge into any branch.

- **Fast-forward:** If `main` has no separate new commits, Git can move its pointer to the feature tip without a merge commit.
- **Three-way merge:** If both branches advanced, a normal merge combines changes since their common ancestor and creates a merge commit.

Your A-F example: `main` has A, B, C, D; `feature-x` has A, B, C, E, F. A normal merge creates M with parents D and F. E and F keep their original ancestry from C; M brings both lines of work together.

Merge conflicts

A conflict occurs when Git cannot automatically combine changes, for example incompatible edits to the same lines.

1. Run `git status` to find conflicted files.
2. Edit them into the intended final version; remove conflict markers (<<<<<<<, =======, >>>>>>>).
3. Stage the resolved files and finish the merge:

```
git add file.txt
git commit
# To cancel an in-progress merge instead:
git merge --abort
```

Rebase = replay commits on a new base

```
git switch feature-x
git rebase main
```

In the same example, rebase creates E′ and F′ after D, giving a linear history: **A → B → C → D → E′ → F′**. Replayed commits have new hashes. `main` stays unchanged until you integrate the branch.

If conflicts occur: fix files → `git add file.txt` → `git rebase --continue`. Cancel with `git rebase --abort`.

Rule: Rebase your own unpublished work freely; coordinate before rewriting history others already use.

4. Stash, undo & tags

Stash = temporarily put unfinished changes aside

Stash saves tracked staged and unstaged changes without adding a normal commit to your branch history. Untracked files require `-u`; ignored files require `-a`.

```
git stash push -u -m "WIP: login feature"
git switch main
# Make the urgent fix, then:
git add bugfix.txt
git commit -m "Fix urgent bug"
git switch feature-x
git stash pop
```

Command	Meaning
`git stash list`	List saved stashes
`git stash show -p`	Inspect the latest stash's tracked changes
`git stash apply`	Restore the latest stash and keep its entry
`git stash pop`	Restore it and remove its entry if successful
`git stash apply --index`	Also attempt to restore the staged state
`git stash drop`	Delete the latest stash entry

Remember: Stashes are local and can be applied on another branch. Restoration may conflict; a failed `pop` keeps the stash. A WIP commit is also a valid way to pause work, and can later be squashed.

Undo: choose by what you want to undo

Command	Effect
`git restore --staged file.txt`	Unstage; keep working edits (in a repo with a commit)
`git restore file.txt`	Discard unstaged tracked edits; restore from the index
`git revert HASH`	Create a new commit undoing an earlier commit; useful for shared history
`git reset --soft HEAD~1`	Move back one commit; keep its changes staged
`git commit --amend`	Replace the latest commit using staged content and an editable message
`git reflog`	Inspect local reference movements to help recover lost commit pointers

Careful: `restore` can discard edits. `reset` and `amend` can rewrite branch history; use the examples on unpublished work. Reflog is not a backup of arbitrary uncommitted edits.

Tags = named markers for commits

```
git tag -a v1.0 -m "Release 1.0"  # Annotated: metadata + message
git tag v1.1                      # Lightweight: simple reference
git tag                          # List tags
git push origin v1.0              # Publish a particular tag
```

Annotated tags suit releases. Tags do not move forward with new commits, and an ordinary branch push does not necessarily publish them. A GitHub Release can add notes and assets around a tag.

5. Remotes, GitHub & publishing

Fetch vs pull vs push

Command	What happens
`git remote -v`	Show configured remote URLs
`git remote add origin URL`	Name a remote `origin`
`git fetch origin`	Download history and update remote-tracking refs such as `origin/main`; leave working files and the current branch unchanged
`git pull --ff-only`	Fetch, then fast-forward; refuse if histories diverged
`git pull --rebase`	Fetch, then rebase local work onto the fetched branch
`git push -u origin main`	Push `main` and set its upstream tracking branch
`git push`	Push according to tracking/configuration; does not upload uncommitted edits

Correction: Pull means fetch + integration (merge or rebase). It does not always create a commit. `origin/main` is your local record of the remote branch, updated by fetch; it is distinct from your local `main`.

Clone, fork & pull request

- **Clone:** A local copy made with Git. Cloning does not grant write permission to the original remote.
- **Fork:** A separate repository under your account on GitHub, linked to the original. Changes pushed to your fork do not change the original repo.
- **Pull request (PR):** Proposes integrating a source branch into a target branch. Reviewers can comment or request changes; an authorized person merges it. Approval alone is not a merge.

Typical contribution: **Fork → clone fork → create branch → edit → commit → push branch → open PR → review → merge.** Push more commits to the same source branch to update an open PR.

Forks do not automatically receive new upstream commits. To sync your fork's `main` when it has no divergent work:

```
git remote add upstream https://github.com/OWNER/REPO.git
git fetch upstream
git switch main
git merge --ff-only upstream/main
git push origin main
```

6. SSH & publishing your notes

You still need access permission. In Git Bash on Windows:

```
ssh-keygen -t ed25519 -C "YOUR_EMAIL"
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
ssh -T git@github.com
git clone git@github.com:USERNAME/REPO.git
```

Between `cat` and `ssh -T`, copy the **public** `.pub` **key** into GitHub → Settings → SSH and GPG keys → New SSH key. Keep the private key private. HTTPS with a token or credential manager is another option.

Publish these notes to a new, empty GitHub repo

Create the repo on GitHub without an initial README, then run these inside the folder containing this `README.md`:

```
git init -b main
git add README.md
git commit -m "Add Git and GitHub revision notes"
git remote add origin https://github.com/YOUR_USERNAME/git-notes.git
git push -u origin main
```

For an existing repo, clone it first, copy this file into it, review the diff, then commit and push. Do not force-push to solve a rejected push.

Official references

- Git command reference
- Git stash · Fetch · Pull
- Rebase · Restore · Ignore rules
- GitHub: Fork a repository
- GitHub: SSH authentication